LOWESS/Spline Smoothing Data Application and Analysis Project

Dataset: Bike Sharing Dataset

Goal: Apply LOWESS and Spline smoothing to this dataset, and analyze their impact on understanding the raw data, and compare their outcomes to each other.

1. Setup

In [52]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from statsmodels.nonparametric.smoothers_lowess import lowess
from scipy.interpolate import UnivariateSpline
```

2. Load Dataset

In [53]:
```
plt.rcParams['figure.figsize'] = (8,5)

df = pd.read_csv('hour.csv')

df = df[['hr','temp','cnt']]
df.dropna(inplace=True)
df
```

Out[53]:

	hr	temp	cnt
0	0	0.24	16
1	1	0.22	40
2	2	0.22	32
3	3	0.24	13
4	4	0.24	1
...	...	...	...
17374	19	0.26	119
17375	20	0.26	89
17376	21	0.26	90
17377	22	0.26	61
17378	23	0.26	49

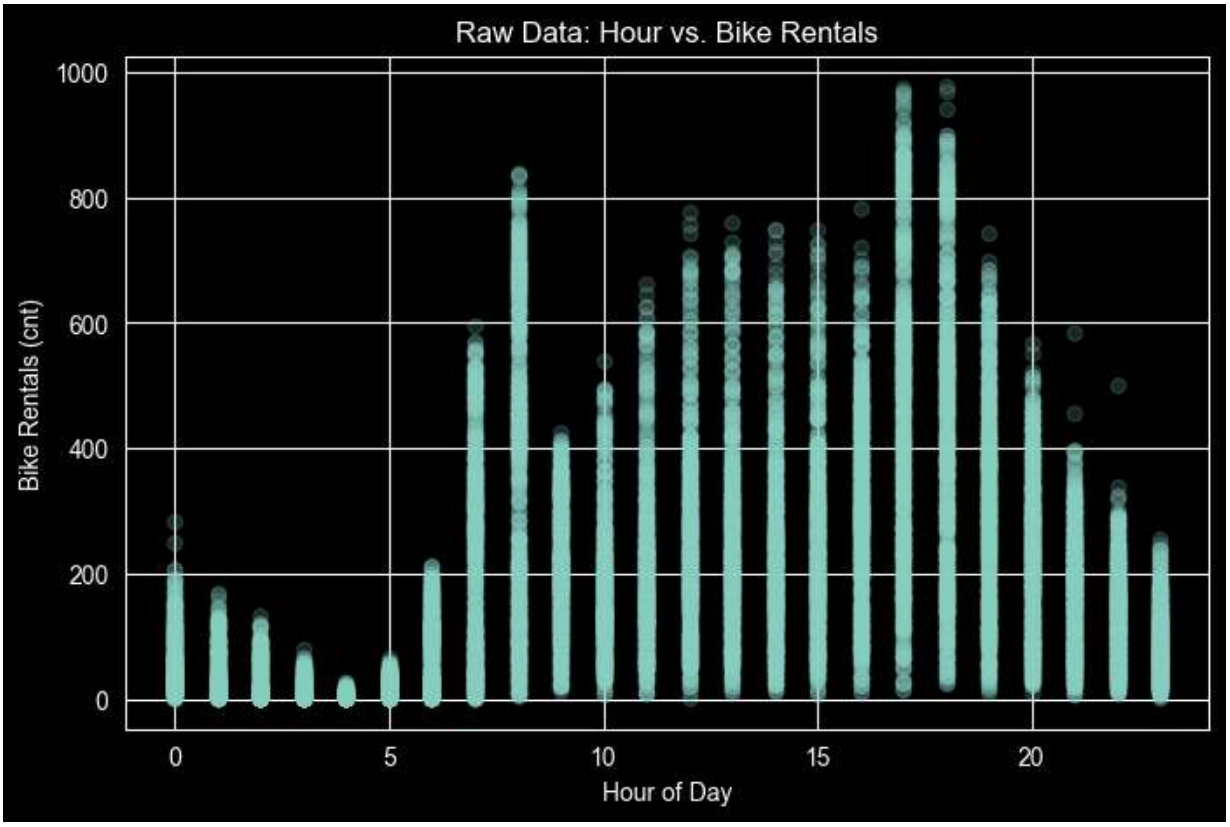
17379 rows × 3 columns

3. Raw Data Visualization

Visualizing Hour vs Number of Bike Rentals

In [54]:

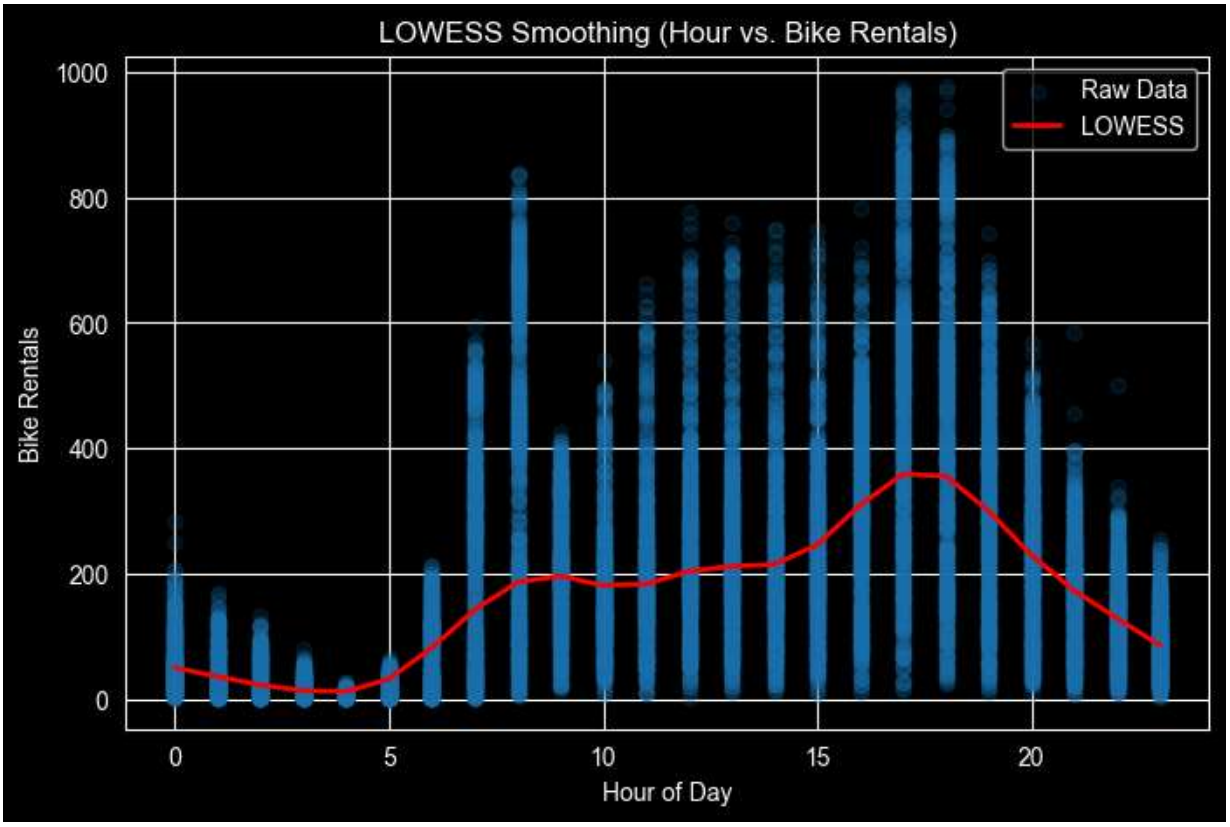
```
plt.scatter(df['hr'],df['cnt'], alpha=0.2)
plt.xlabel('Hour of Day')
plt.ylabel('Bike Rentals (cnt)')
plt.title('Raw Data: Hour vs. Bike Rentals')
plt.show()
```



4. Lowess Smoothing

Smoothing hr vs. count

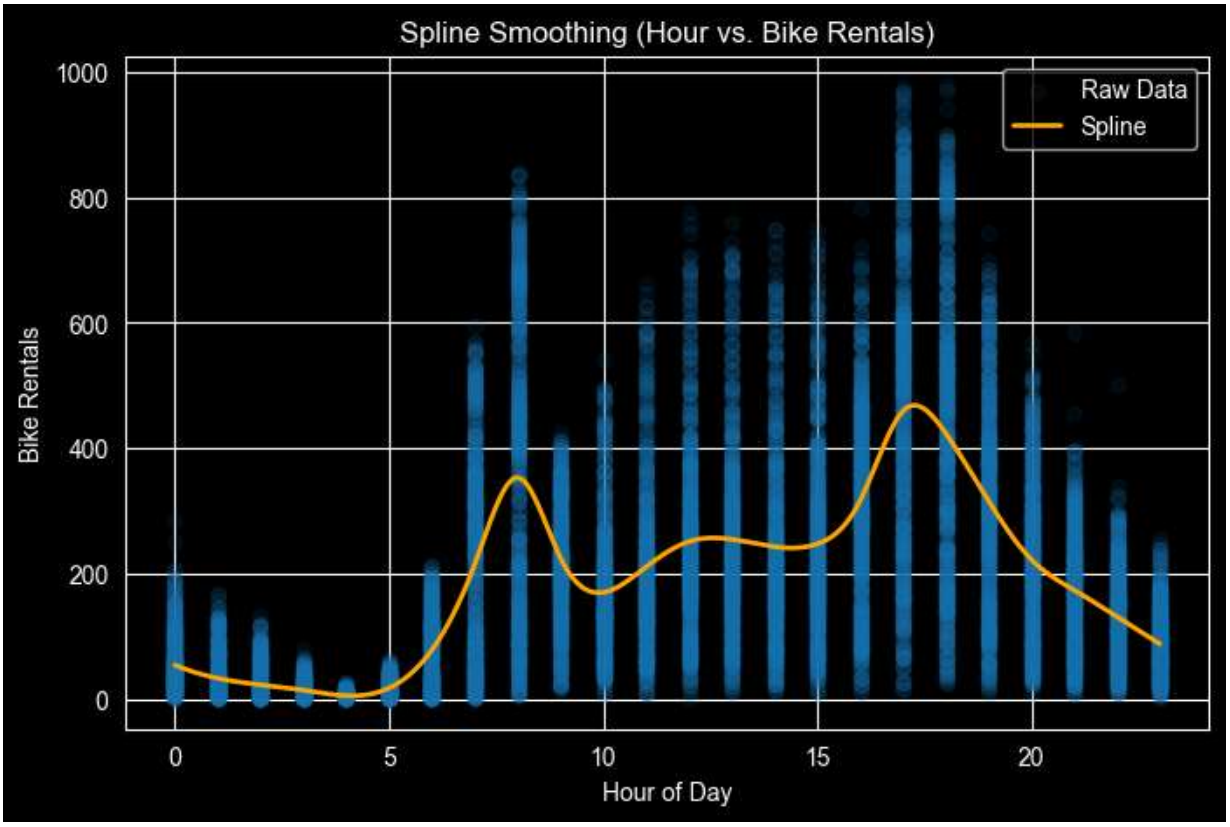
In [55]:
```
#set frac to 0.2, which means each local regression uses 20% of the nearest data po
#lowering frac increases flexibility
#lowess function returns sorted (X,Y) pairs, which are then plotted later
lowess_hr = lowess(df['cnt'], df['hr'], frac=0.2)

#plot scatter
plt.scatter(df['hr'],df['cnt'], alpha=0.15, color='tab:blue', label='Raw Data')

#plot lowess output
plt.plot(lowess_hr[:,0],lowess_hr[:,1], color='red', linewidth=2, label='LOWESS')

plt.xlabel('Hour of Day')
plt.ylabel('Bike Rentals')
plt.title('LOWESS Smoothing (Hour vs. Bike Rentals)')
plt.legend()
plt.show()
```



5. Spline Smoothing

Spline Smoothing for Hour vs. Num Bike Rentals

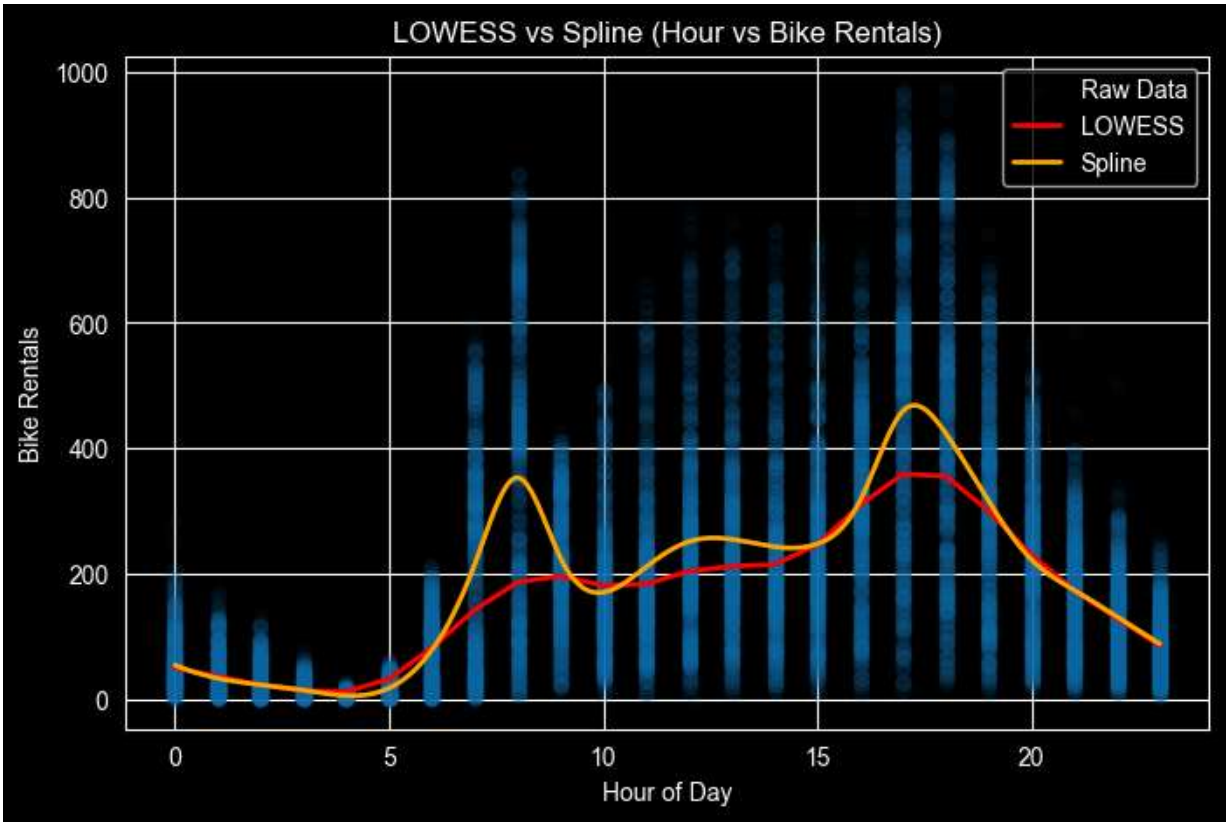
In [56]:
```
# aggregate data for spline
df_grouped = df.groupby('hr')['cnt'].mean().reset_index()

x = df_grouped['hr']
y = df_grouped['cnt']

# fit spline
from scipy.interpolate import UnivariateSpline
spline = UnivariateSpline(x, y, s=200)

# generate curve
x_range = np.linspace(x.min(), x.max(), 1000)
y_spline = spline(x_range)

# plot
plt.scatter(df['hr'], df['cnt'], alpha=0.1, color='tab:blue', label='Raw Data')
plt.plot(x_range, y_spline, color='orange', linewidth=2, label='Spline')

plt.xlabel('Hour of Day')
plt.ylabel('Bike Rentals')
plt.title('Spline Smoothing (Hour vs. Bike Rentals)')
plt.legend()
plt.show()
```



6. Direct Comparison

Spline vs. LOWESS

In [57]:
```
plt.scatter(df['hr'], df['cnt'], alpha=0.05, color='tab:blue', label='Raw Data')

plt.plot(lowess_hr[:, 0], lowess_hr[:, 1],
         color='red', linewidth=2, label='LOWESS')

plt.plot(x_range, y_spline,
         color='orange', linewidth=2, label='Spline')

plt.xlabel('Hour of Day')
plt.ylabel('Bike Rentals')
plt.title('LOWESS vs Spline (Hour vs Bike Rentals)')
plt.legend()
plt.show()
```



7. Key Takeaways

LOWESS	Spline
LOWESS smoothing is flatter than Spline smoothing	Spline smoothing is much more sensitive than LOWESS smoothing, and can require some tuning with the smoothing parameter to avoid overrepresenting outliers
LOWESS smoothing captures the overall trend of the data, identifying peak hours are 16-19, and not being as vulnerable to the 8am spike	Spline smoothing captures local trends better, identifying the 8am bike rental spike, the 12-14 bump in bike usage, and the peak hours of 16-19
LOWESS would mostly be used to identify broader trends in bike usage throughout the hours of the day. Good for stakeholders and researchers doing Exploratory Data Analysis (EDA)	Spline smoothing would mostly be used for making predictions and building models. Spline smoothing preserves more accurate data on the hour-by-hour level, and can help a planner identify how many bikes would be in use at 2:30